CP2/CP3 · Generative AI for Engineering · 4ESPY-2026

Sentimento por aspecto em avaliações de produtos com Qwen3-1.7B + LoRA

Fine-tuning de uma LLM local em GPU (Apple M5 · Metal/MPS), comparação de hiperparâmetros, avaliação das respostas e frontend para o usuário criar o prompt.

Professor: Francisco Elanio

Integrantes do grupo

Nome	RM
Felipe Cortez	99750
Julia Lins	98690
Luis Barreto	99210
Victor Aranda	99667
Guilherme Akio	98582

Repositório: https://github.com/lgustavobarre351/GENERATIVE_AI_CP2-CP3

Data de entrega: 15/10/2026

Resultado principal	Modelo sem treino	Modelo treinado (LoRA)
Acerto exato · 400 avaliações com frases inéditas	25,5%	**83,0%**
F1 dos pares (aspecto, sentimento)	0,514	**0,926**
Acerto exato · 40 avaliações escritas à mão	27,5%	**82,5%**

Sumário

1. Requisitos da atividade e onde estão atendidos
2. Problema escolhido
3. LLM escolhida e técnica de treino
4. Dados
5. Como o sistema funciona
6. Ambiente e GPU
7. Experimentos (parâmetros testados)
8. Metodologia de avaliação
9. Resultados
10. Conclusões sobre os parâmetros
11. Avaliação das respostas
12. Frontend
13. Como executar
14. Estrutura do repositório
15. Limitações e próximos passos
16. Referências

Apêndice A. Código-fonte completo

1. Requisitos da atividade e onde estão atendidos

Requisito do enunciado	Como foi atendido	Seção
Treinar uma LLM local	Qwen3-1.7B treinado com LoRA localmente, sem nenhuma API externa	3, 7
Selecionar/escolher a LLM	Qwen3-1.7B (lista sugerida pelo professor), com justificativa	3
Escolher um problema específico	Sentimento por aspecto (lista sugerida pelo professor)	2
Testar diferentes parâmetros	7 configurações: rank do LoRA, learning rate, épocas e camadas, mais o baseline	7, 9, 10
Verificar/avaliar o modelo (respostas)	6 métricas em 2 conjuntos de teste, matrizes de confusão e as respostas de cada exemplo	8, 9, 11
Fazer todos os testes utilizando GPU	Todos os treinos e avaliações na GPU do Apple M5 (Metal/MPS), autorizada pelo professor	6
Criar um frontend para o usuário inserir/criar o prompt	App Gradio com 4 abas, incluindo 'Prompt livre'	12
Print da GPU instalada e funcionando no VSCode	Print com informações da GPU e monitor a 98% de uso	6
Documentação e código no GitHub	https://github.com/lgustavobarre351/GENERATIVE_AI_CP2-CP3 (código completo também no Apêndice A)	13, 14, A
Grupo de 4 a 5 pessoas	5 integrantes	Capa

2. Problema escolhido: sentimento por aspecto

Problema da lista sugerida pelo professor (categoria *Classificação*). Dada uma avaliação de produto escrita em português, o modelo identifica o sentimento de **cada aspecto mencionado** e responde em JSON:

```
"Entrega rápida, mas o produto veio arranhado e achei caro."
→ {"entrega": "positivo", "qualidade": "negativo", "preço": "negativo"}
```

- **Aspectos:** entrega, qualidade, preço, atendimento, embalagem.
- **Sentimentos:** positivo, negativo, neutro.
- Aspectos não mencionados ficam fora do JSON. Sem nenhum aspecto, a resposta é `{}`.

Por que esse problema? Um sentimento geral esconde informação: uma avaliação de 3 estrelas pode elogiar o produto e reclamar da transportadora. Separar por aspecto mostra **onde** está o problema (logística, fornecedor, preço, SAC, embalagem). Além disso, a resposta tem formato fixo e é **verificável automaticamente**, campo a campo, o que permite comparar configurações de treino de forma objetiva.

3. LLM escolhida e técnica de treino

Critério	Qwen3-1.7B
Lista do professor	Qwen3 (0.6B, **1.7B**, 4B, 8B)
Português	Modelo multilíngue, com bom desempenho em PT-BR
Tamanho	1,7 bilhão de parâmetros: cabe e treina com LoRA na GPU de um notebook (Apple M5, 16 GB)
Formato de saída	Segue instruções e gera JSON. O modo de raciocínio (`<think>`) foi desligado para respostas diretas
Licença	Apache 2.0

LoRA (Low-Rank Adaptation). Em vez de atualizar os 1,7 bilhão de pesos, o modelo original fica congelado e treinamos pequenas matrizes de baixo rank inseridas nas camadas lineares. Na configuração final (r=16), são 17.4 milhões de parâmetros treináveis (1.0% do total), e o adaptador resultante tem 33 MB.

4. Dados

Seguindo a sugestão do professor, os dados são **sintéticos e com verificação exata**, gerados por `src/data.py`:

- Banco de 184 frases escritas à mão (5 aspectos × 3 sentimentos), combinadas com conectivos de contraste ("mas", "porém", "por outro lado"...).
- Frases sem aspecto ("Comprei para dar de presente.", "Recomendo!"), que o modelo deve ignorar.
- Ruído de texto real: 25% das avaliações em minúsculas e 15% sem acentos.
- Cada avaliação tem de 0 a 4 aspectos.

Conjunto	Exemplos	Observação
Treino	2000	Frases do "banco de treino" (75% das frases de cada aspecto/sentimento)
Validação	200	Mesmo banco do treino; usado para a loss de validação
Teste	400	**Somente frases que nunca aparecem no treino** (25% restantes)
Teste manual	40	Escritas à mão, com linguagem natural, gírias e abreviações ("tá", "pq", "mt"), fora do gerador

O teste manual mede se o modelo generaliza para avaliações reais, e não apenas para o padrão do gerador. Exemplos do conjunto de treino:

Avaliação	Resposta esperada (JSON)
A embalagem é discreta; no entanto, a bateria não dura nada, porém a entrega foi no prazo normal. Vou usar na casa de praia.	`{"entrega": "neutro", "qualidade": "negativo", "embalagem": "neutro"}`
veio muito bem embalado. além disso, fui muito bem atendido. estava precisando de um desses.	`{"atendimento": "positivo", "embalagem": "positivo"}`
o frete foi rapidissimo. pesquisei bastante antes de comprar.	`{"entrega": "positivo"}`
A caixa chegou toda amassada, o vendedor foi grosseiro, só que o frete foi rapidíssimo!	`{"entrega": "positivo", "atendimento": "negativo", "embalagem": "negativo"}`
É a segunda vez que compro. Chegou antes do prazo. A embalagem é reforçada, com plástico bolha. A qualidade superou minhas expectativas, comprei em promoção e valeu muito a pena.	`{"entrega": "positivo", "qualidade": "positivo", "preço": "positivo", "embalagem": "positivo"}`
A caixa chegou intacta.	`{"embalagem": "positivo"}`

5. Como o sistema funciona

`avaliação → prompt (system + chat template do Qwen3) → Qwen3-1.7B + LoRA → JSON {aspecto: sentimento}`

- **Prompt:** o *system prompt* lista os aspectos, os sentimentos e o formato, com um exemplo. O mesmo prompt é usado no modelo sem treino (baseline), para a comparação ser justa.
- **Treino:** a loss é calculada **somente nos tokens da resposta** (o JSON); o prompt é mascarado.
- **Inferência:** geração determinística (greedy) em lotes. A resposta passa por um parser que valida o JSON e descarta aspectos ou sentimentos fora da lista.

System prompt utilizado:

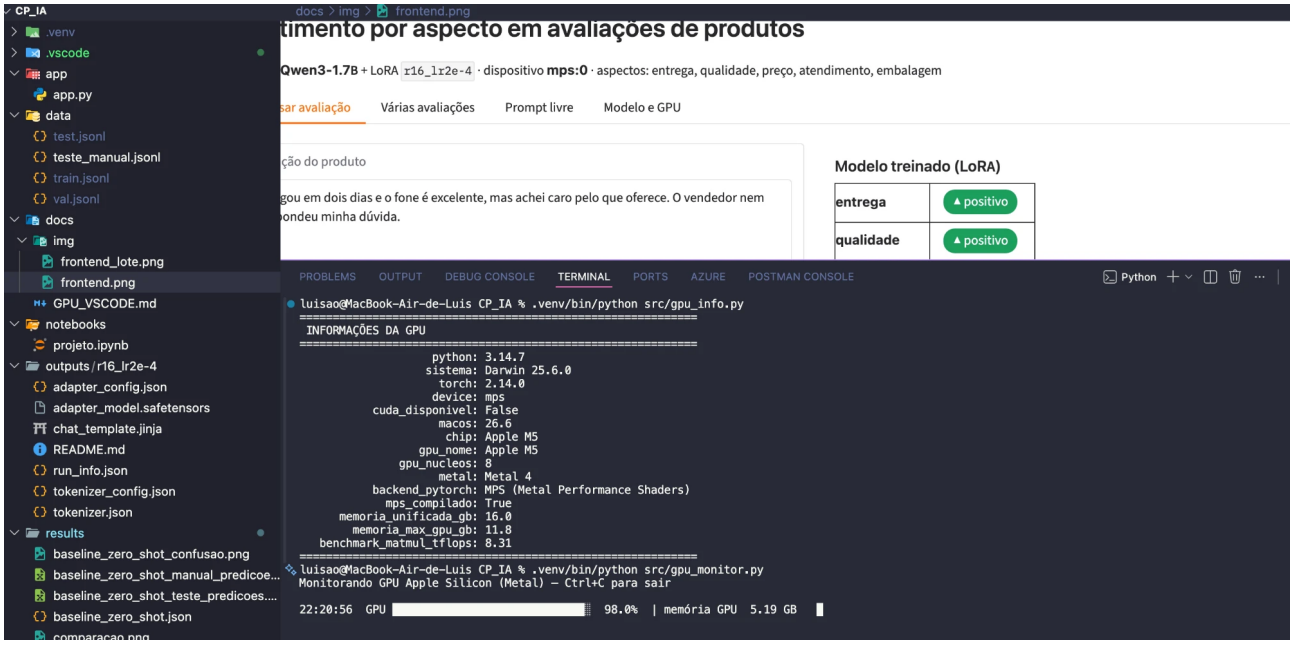
```
Você analisa avaliações de produtos escritas em português e identifica o sentimento de cada
aspecto mencionado.
Aspectos possíveis: entrega, qualidade, preço, atendimento, embalagem.
Sentimentos possíveis: positivo, negativo, neutro.
Responda somente com um JSON em que cada chave é um aspecto mencionado e o valor é o sentimento
dele. Inclua apenas os aspectos mencionados na avaliação. Se nenhum aspecto for mencionado,
responda {}.
Exemplo: "Paguei barato, mas a tampa quebrou." -> {"qualidade": "negativo", "preço": "positivo"}
```

Observação: na primeira versão, o prompt dizia apenas `{"aspecto": "sentimento"}`, e o modelo base interpretou literalmente, respondendo `{"aspecto": "entrega", "sentimento": ...}`. Um exemplo concreto foi adicionado ao prompt para o baseline ser avaliado de forma justa (100% de JSON válido).

6. Ambiente e GPU

Todos os treinos e avaliações foram executados na **GPU integrada do Apple M5**, via backend **MPS (Metal Performance Shaders)** do PyTorch. O uso de Metal no lugar de CUDA foi autorizado pelo professor, já que o computador do grupo não tem placa NVIDIA. O código também roda em CUDA sem alterações: `src/config.py` escolhe automaticamente CUDA → MPS → CPU.

Item	Valor
Chip / GPU	Apple M5 · 8 núcleos de GPU
API gráfica	Metal 4
Backend PyTorch	MPS (Metal Performance Shaders)
Memória unificada	16 GB (até 11.8 GB disponíveis para a GPU)
Benchmark (multiplicação de matrizes fp16)	12.45 TFLOPS
Sistema	macOS 26.6
Software	Python 3.14.7 · PyTorch 2.14.0 · Transformers 5 · PEFT 0.21



Print da GPU instalada e funcionando dentro do VSCode: à esquerda, o projeto; em cima, o frontend aberto; embaixo, a saída de src/gpu_info.py e o monitor ao vivo (src/gpu_monitor.py) com a GPU a 98%.

Metal × CUDA

	NVIDIA	Apple Silicon (usado)
API da GPU	CUDA + cuDNN	Metal + Metal Performance Shaders
Dispositivo no PyTorch	torch.device("cuda")	torch.device("mps")
Instalação	Driver NVIDIA + PyTorch com a versão de CUDA compatível	macOS 12.3+ e o PyTorch padrão do PyPI
Memória	VRAM dedicada	Memória unificada (CPU e GPU compartilham os 16 GB)
Monitoramento no VSCode	nvidia-smi, extensões GPU Monitor / GPU Environment	src/gpu_monitor.py (lê o driver via ioreg)

As extensões GPU Monitor e GPU Environment dependem do `nvidia-smi` e só funcionam com NVIDIA. Por isso, foi criado o `src/gpu_monitor.py`, que mostra a utilização e a memória da GPU ao vivo no terminal do VSCode. Ele também detecta NVIDIA e passa a usar o nvidia-smi.

Otimizações de desempenho aplicadas (aprendidas durante a execução)

Problema observado	Solução	Efeito
Treino em fp32 muito lento	Modelo base em bf16, pesos do LoRA em fp32	4× mais rápido, mesma loss
Batch 8 fazia o Mac usar swap	Batch 4 + acumulação de gradiente 4 (batch efetivo 16)	2,7× mais rápido
Velocidade caía de 13 para 42 s/passo ao longo do treino	Padding em múltiplos de 32 e limpeza do cache do MPS a cada 10 passos	Estável em ~14 s/passo
Sem gradient checkpointing, a memória estourava	Gradient checkpointing ligado	Cabe nos ~11,8 GB da GPU
Na bateria, o Mac suspendeu e o treino parou	Treino na tomada; experimentos retomáveis (pulam o que já terminou)	Nenhuma perda de progresso

7. Experimentos (parâmetros testados)

Configuração de referência: LoRA r=16, α=32, dropout 0,05, learning rate 2e-4, 1 época, LoRA em todas as camadas lineares (q, k, v, o, gate, up, down), batch efetivo 16, scheduler cosseno com 5% de warmup, bf16, gradient checkpointing. Cada experimento muda **um único parâmetro** em relação à referência, para isolar o seu efeito.

Experimento	Parâmetro alterado	Hipótese
baseline_zero_shot	Sem treino	Ponto de partida
r16_lr2e-4	Referência	—
r4_lr2e-4 / r64_lr2e-4	Rank do LoRA: 4 / 64	Rank maior = mais capacidade e mais parâmetros treináveis
r16_lr2e-5 / r16_lr1e-3	Learning rate 10× menor / 5× maior	LR baixo aprende pouco; LR alto pode ficar instável
r16_lr2e-4_ep2	2 épocas	Mais épocas podem melhorar ou causar overfitting
r16_lr2e-4_attn	LoRA só na atenção (q, k, v, o)	Menos parâmetros; mede a importância das camadas MLP

8. Metodologia de avaliação

Métrica	O que mede
JSON válido	% de respostas no formato correto
Acerto exato	% de avaliações com **todos** os aspectos e sentimentos certos, sem aspectos a mais
F1 dos pares (principal)	F1 micro dos pares (aspecto, sentimento)
F1 de detecção	Se o modelo encontra os aspectos mencionados, ignorando o sentimento
Acurácia de polaridade	Entre os aspectos detectados, quantos sentimentos estão certos
F1 por aspecto e matriz de confusão	Onde o modelo erra

As respostas de cada modelo para cada exemplo ficam salvas em `results/<experimento>_<conjunto>_predicoes.csv` (texto, esperado, resposta crua do modelo, previsto e se acertou), para a análise qualitativa da seção 11.

9. Resultados

Baseline × melhor modelo

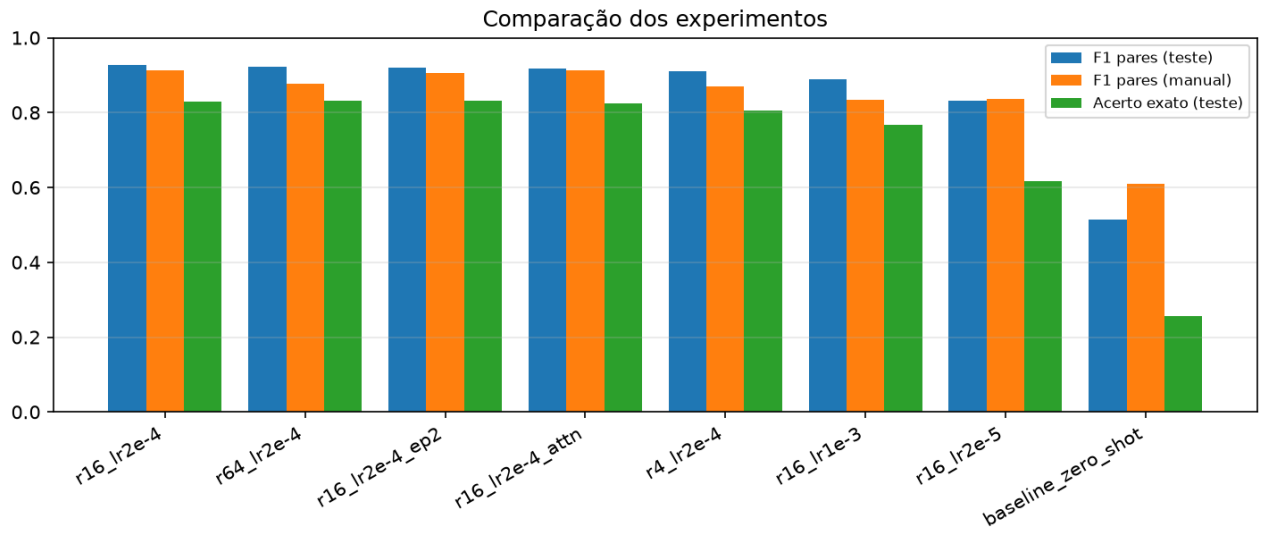
Métrica	Qwen3-1.7B sem treino	Qwen3-1.7B + LoRA (r16_lr2e-4)
JSON válido (teste)	100%	100%
Acerto exato (teste, 400 frases inéditas)	25,5%	**83,0%**
F1 dos pares (teste)	0,514	**0,926**
F1 de detecção de aspectos (teste)	0,707	0,957
Acurácia de polaridade (teste)	72,7%	96,8%
Acerto exato (manual, 40 escritas à mão)	27,5%	**82,5%**
F1 dos pares (manual)	0,611	**0,913**
Tempo de inferência	325 ms/avaliação	365 ms/avaliação

O fine-tuning **mais que triplicou o acerto exato** (25,5% → 83,0%), e o ganho se mantém no teste manual, com linguagem bem diferente da usada no treino. Ou seja, o modelo não apenas decorou as frases do gerador.

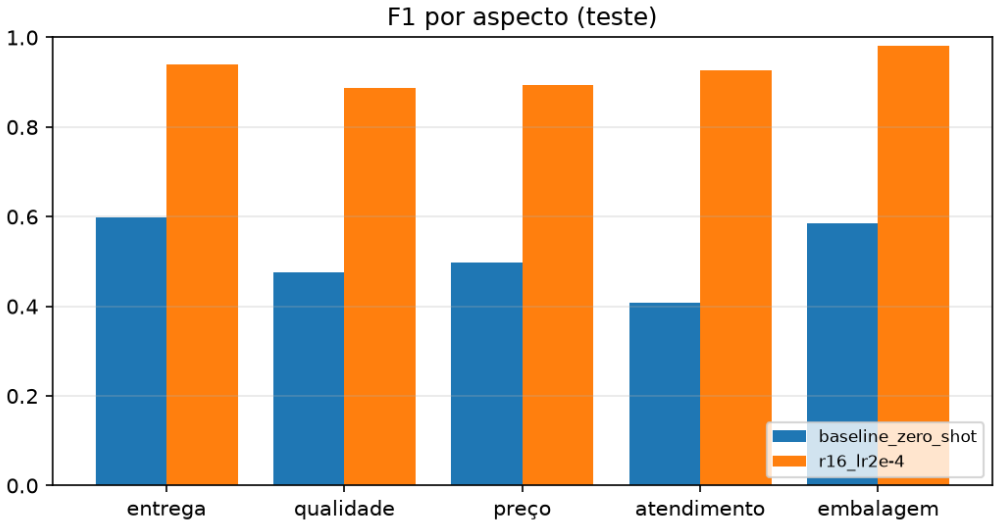
Todos os experimentos

Experimento	r	Camadas	LR	Ép.	Params	Treino	Loss val	Acerto exato	F1 teste	F1 manual	Acerto manual
r16_lr2e-4	16	todas	2e-4	1	17.4M	32 min	0.0004	**83,0%**	**0,926**	**0,913**	**82,5%**
r64_lr2e-4	64	todas	2e-4	1	69.7M	32 min	0.0005	83,2%	0,923	0,878	77,5%
r16_lr2e-4_ep2	16	todas	2e-4	2	17.4M	64 min	0.0002	83,2%	0,920	0,905	82,5%
r16_lr2e-4_attn	16	atenção	2e-4	1	6.4M	25 min	0.0055	82,5%	0,919	0,913	85,0%
r4_lr2e-4	4	todas	2e-4	1	4.4M	29 min	0.0062	80,5%	0,911	0,870	75,0%
r16_lr1e-3	16	todas	1e-3	1	17.4M	32 min	0.0005	76,8%	0,889	0,835	67,5%
r16_lr2e-5	16	todas	2e-5	1	17.4M	30 min	0.0619	61,8%	0,831	0,837	65,0%
baseline_zero_shot	—	—	—	—	—	—	—	25,5%	0,514	0,611	27,5%

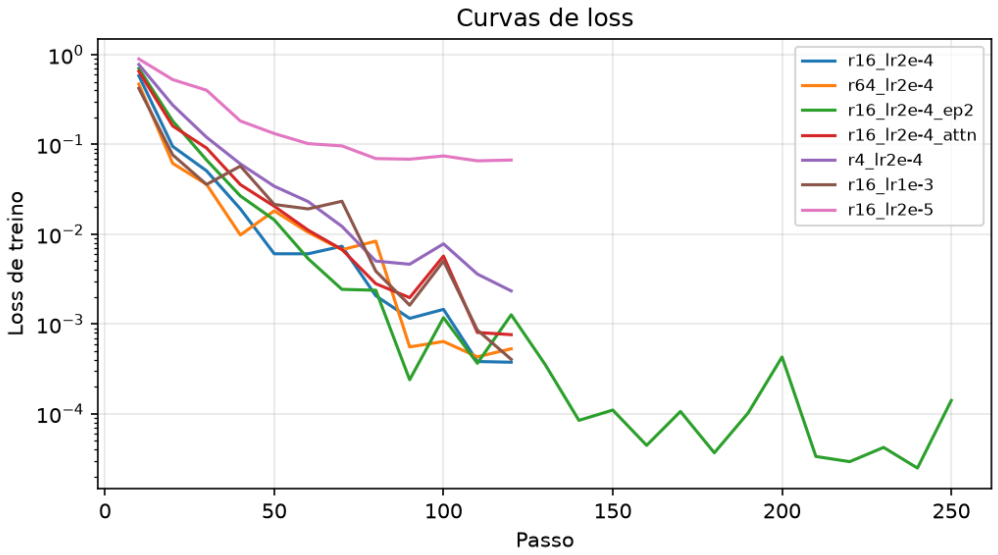
Ordenado pelo F1 dos pares no teste. Tempos de treino na GPU do Apple M5.



Comparação dos experimentos nos dois conjuntos de teste.

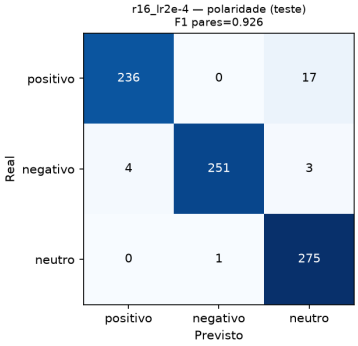


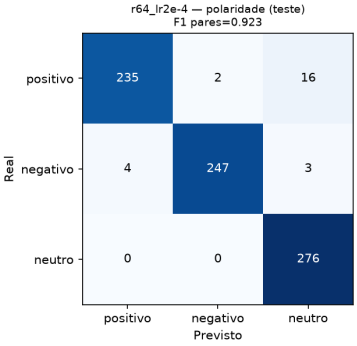
F1 por aspecto: modelo sem treino × melhor modelo.

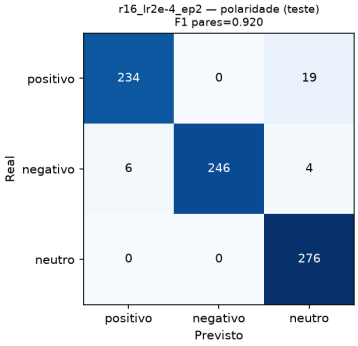


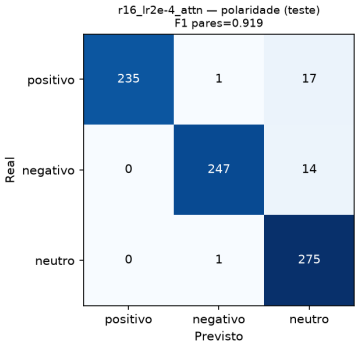
Curvas de loss de treino (escala logarítmica).

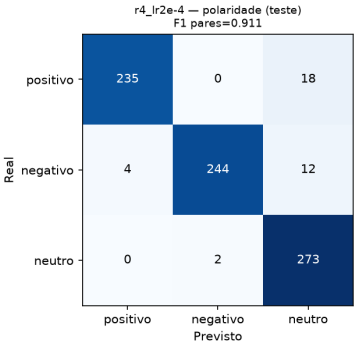
Matrizes de confusão da polaridade (conjunto de teste)

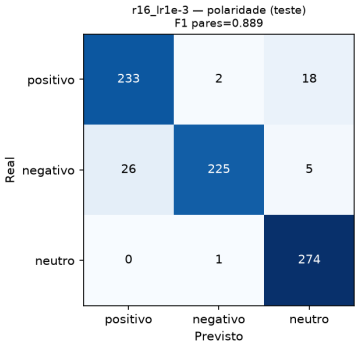


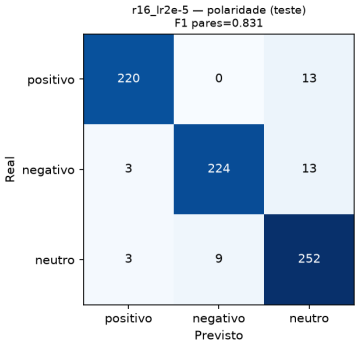


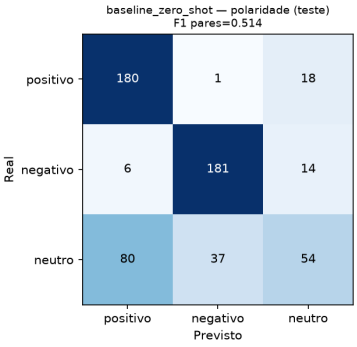












10. Conclusões sobre os parâmetros

- **Learning rate foi o parâmetro mais importante.** Com 2e-5, o modelo mal aprendeu em 1 época (loss de validação 100× maior, acerto exato de 61,8%). Com 1e-3, a loss caiu tanto quanto na referência, mas a generalização piorou (F1 manual de 0,835): o modelo se ajustou demais ao padrão do gerador. **2e-4 foi o melhor equilíbrio.**
- **Rank do LoRA:** r=4 tem pouca capacidade (F1 0,911). r=64 tem 4× mais parâmetros que r=16 e não melhora no teste, além de generalizar pior no teste manual (0,878). **r=16 é o ponto ideal.**
- **Épocas:** a 2ª época dobrou o tempo e reduziu a loss de validação, mas não melhorou o teste. É sinal de que, depois da 1ª época, o modelo passa a decorar o conjunto de treino.
- **Camadas:** LoRA só na atenção, com 37% dos parâmetros e o treino mais rápido (25 min), ficou praticamente empatado com a referência e teve o melhor acerto exato no teste manual (85%). É uma boa opção quando o tempo de GPU é limitado.
- **Loss de validação baixa não garante generalização:** r16_lr1e-3 e r16_lr2e-4_ep2 tiveram as menores losses e não foram os melhores. Por isso a escolha do modelo foi feita pelas métricas de teste, e não pela loss.

11. Avaliação das respostas

Abaixo estão as respostas do modelo sem treino e do modelo treinado para **todas as 40 avaliações do teste manual**. O modelo treinado errou 7, e o modelo sem treino errou 29. ✓ = resposta idêntica à esperada; ✗ = diferente.

#	Avaliação	Esperado	Sem treino		Treinado	
1	Chegou em 3 dias, bem antes do que o site falava. O fone é bom demais, grave forte e confortável.	entrega: pos, qualidade: pos	entrega: pos, qualidade: pos	✓	entrega: pos, qualidade: pos	✓
2	Produto lindo mas a caixa veio toda detonada, sorte que não quebrou nada.	qualidade: pos, embalagem: neg	embalagem: neg	✗	qualidade: pos, embalagem: neg	✓
3	Paguei quase 500 reais nisso e em um mês já tá com mau contato. Não compensa.	qualidade: neg, preço: neg	qualidade: neg, preço: pos	✗	qualidade: neg, preço: neg	✓
4	Liguei 4 vezes no SAC pra trocar e ninguém resolveu, um descaso total.	atendimento: neg	atendimento: neg	✓	atendimento: neg	✓
5	Melhor compra do ano, baratinho e funciona que é uma beleza	qualidade: pos, preço: pos	preço: pos	✗	qualidade: pos, preço: pos	✓
6	demorou 20 dias pra chegar, mas o produto é exatamente como na foto, muito bom	entrega: neg, qualidade: pos	entrega: neg, qualidade: pos	✓	entrega: neg, qualidade: pos	✓
7	Comprei pra minha mãe no dia das mães.	{}	entrega: neu	✗	{}	✓
8	A vendedora tirou todas as minhas dúvidas pelo whatsapp, super educada. Chegou certinho na data combinada.	entrega: neu, atendimento: pos	entrega: pos, atendimento: pos	✗	entrega: neu, atendimento: pos	✓
9	O preço tava igual em todo lugar, comprei aqui pq tinha frete grátis e chegou rápido.	entrega: pos, preço: neu	entrega: pos	✗	entrega: pos, preço: neu	✓
10	veio dentro de um saquinho plástico sem nada de proteção, o vidro rachou	qualidade: neg, embalagem: neg	embalagem: neg	✗	qualidade: neg, embalagem: neg	✓
11	Liquidificador fraco, não tritura gelo direito. Pelo menos foi barato.	qualidade: neg, preço: pos	qualidade: neg, preço: pos	✓	qualidade: neg, preço: pos	✓
12	Entrega no prazo, produto ok. Nada a declarar.	entrega: neu, qualidade: neu	entrega: pos, qualidade: pos	✗	entrega: pos, qualidade: neu	✗
13	O rastreamento dizia entregue mas não recebi nada, e o atendimento só manda mensagem automática.	entrega: neg, atendimento: neg	entrega: neg, atendimento: neg	✓	entrega: neg, atendimento: neu	✗

#	Avaliação	Esperado	Sem treino		Treinado	
14	Que caixa bonita! Dei de presente e todo mundo elogiou a apresentação.	embalagem: pos	embalagem: pos	✓	embalagem: pos	✓
15	custo beneficio otimo, recomendo pra quem quer gastar pouco	preço: pos	{}	✗	preço: pos	✓
16	A cadeira é confortável, montagem fácil, material resistente. Entregue em uma semana, o normal.	entrega: neu, qualidade: pos	qualidade: pos	✗	entrega: neu, qualidade: pos	✓
17	Tive problema com a nota fiscal e o pessoal do suporte resolveu em minutos. Nota mil pro atendimento.	atendimento: pos	atendimento: pos	✓	atendimento: pos	✓
18	Esperava mais pelo valor que paguei. A qualidade é só mediana.	qualidade: neu, preço: neg	qualidade: neg	✗	qualidade: neu, preço: neg	✓
19	Bateria do celular não aguenta meio dia. Horrível.	qualidade: neg	{}	✗	qualidade: neg	✓
20	Chegou super bem embalado, com isopor e plástico bolha, e antes do prazo ainda!	entrega: pos, embalagem: pos	embalagem: pos	✗	entrega: pos, embalagem: pos	✓
21	O tênis é lindo, mas o tamanho veio errado e a loja não quis trocar.	qualidade: pos, atendimento: neg	embalagem: neu	✗	qualidade: pos, atendimento: neg	✓
22	Preço justo, qualidade ok, entrega normal. É o que é.	entrega: neu, qualidade: neu, preço: pos	preço: pos	✗	entrega: neu, qualidade: neu, preço: neu	✗
23	Simplesmente parou de ligar depois de 10 dias de uso.	qualidade: neg	entrega: neu	✗	qualidade: neg	✓
24	a entrega atrasou mt e quando chegou a caixa tava aberta	entrega: neg, embalagem: neg	entrega: neg	✗	entrega: neg, embalagem: neg	✓
25	Comprei na promoção relâmpago, saiu pela metade do preço. O aspirador é potente e silencioso.	qualidade: pos, preço: pos	qualidade: pos, preço: pos	✓	qualidade: pos, preço: pos	✓
26	Fiz a compra pelo app, foi tranquilo.	{}	entrega: neu	✗	atendimento: neu	✗
27	O chat demorou um pouco pra responder mas resolveram meu problema. Produto chegou bem embalado.	atendimento: pos, embalagem: pos	entrega: pos, embalagem: pos	✗	atendimento: neu, embalagem: pos	✗
28	Muito caro pro que é. A panela gruda tudo.	qualidade: neg, preço: neg	qualidade: neg, preço: neg	✓	qualidade: neg, preço: neg	✓
29	Chegou rápido, mas veio a cor errada.	entrega: pos, qualidade: neg	entrega: pos, qualidade: neg, embalagem: neg	✗	entrega: pos, qualidade: neg	✓
30	produto mediano, preço de mercado, nada demais	qualidade: neu, preço: neu	qualidade: neu, preço: neu	✓	qualidade: neu, preço: neu	✓
31	Atendimento impecável desde a venda até a entrega, que foi super rápida.	entrega: pos, atendimento: pos	atendimento: pos	✗	entrega: pos, atendimento: pos	✓
32	Embalagem comum de papelão, chegou sem avarias. O relógio é bonito e funciona bem.	qualidade: pos, embalagem: neu	qualidade: pos	✗	qualidade: pos, embalagem: neu	✓
33	Não chegou até hoje e já se passaram 30 dias. Ninguém me dá retorno.	entrega: neg, atendimento: neg	entrega: neg	✗	entrega: neg, atendimento: neg	✓
34	Ótimo! Uso todos os dias.	qualidade: pos	entrega: neu, qualidade: pos, preço: neu, atendimento: neu, embalagem: neu	✗	{}	✗
35	O valor é alto, mas a qualidade compensa, é muito durável.	qualidade: pos, preço: neg	qualidade: pos, preço: neg	✓	qualidade: pos, preço: neg	✓
36	Meu marido gostou.	{}	entrega: neu, qualidade: neu, preço: neu, atendimento: neu, embalagem: neu	✗	{}	✓

#	Avaliação	Esperado	Sem treino		Treinado	
37	Prazo de entrega longo, 15 dias úteis, mas cumpriram. O livro veio impecável.	entrega: neu, qualidade: pos	entrega: neg, qualidade: pos	✗	entrega: neg, qualidade: pos	✗
38	Embalagem rasgada, produto amassado e o vendedor sumiu.	qualidade: neg, atendimento: neg, embalagem: neg	embalagem: neg	✗	qualidade: neg, atendimento: neg, embalagem: neg	✓
39	Barato, chegou rápido e funciona. Sem do que reclamar.	entrega: pos, qualidade: pos, preço: pos	entrega: pos, preço: pos	✗	entrega: pos, qualidade: pos, preço: pos	✓
40	O suporte me atendeu normalmente, seguiram o procedimento padrão de troca.	atendimento: neu	atendimento: pos	✗	atendimento: neu	✓

pos = positivo · neg = negativo · neu = neutro

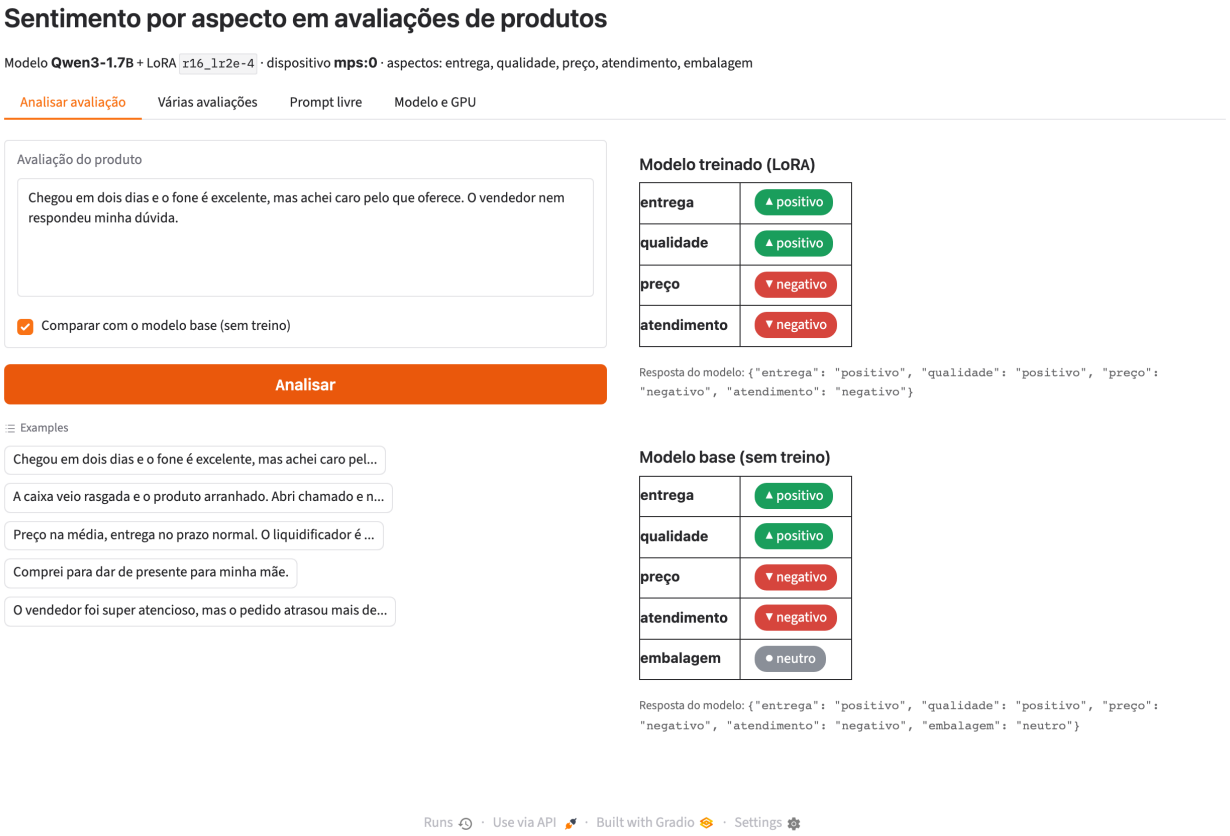
Análise dos erros

- **Modelo sem treino:** omite aspectos claros ("Bateria do celular não aguenta meio dia" → {}), troca polaridade ("Paguei quase 500 reais... Não compensa" → preço: positivo) e lista os 5 aspectos como neutro em avaliações sem nenhum aspecto ("Meu marido gostou.").
- **Modelo treinado:** os 7 erros são, na maioria, casos ambíguos na fronteira do "neutro" ("Entrega no prazo, produto ok" → entrega positivo; "Prazo de entrega longo, mas cumpriram" → entrega negativo).
- **Armadilhas de vocabulário no teste sintético:** a frase "atende o básico" (sobre qualidade) causou 31 dos 34 casos de "qualidade omitida": o modelo associa a palavra "atende" ao aspecto atendimento. Da mesma forma, "é caro demais pelo que **entrega**" faz o modelo marcar também o aspecto entrega. Ou seja, em alguns casos o modelo ainda se apoia em palavras-chave; mais variedade de frases no treino seria o próximo passo.

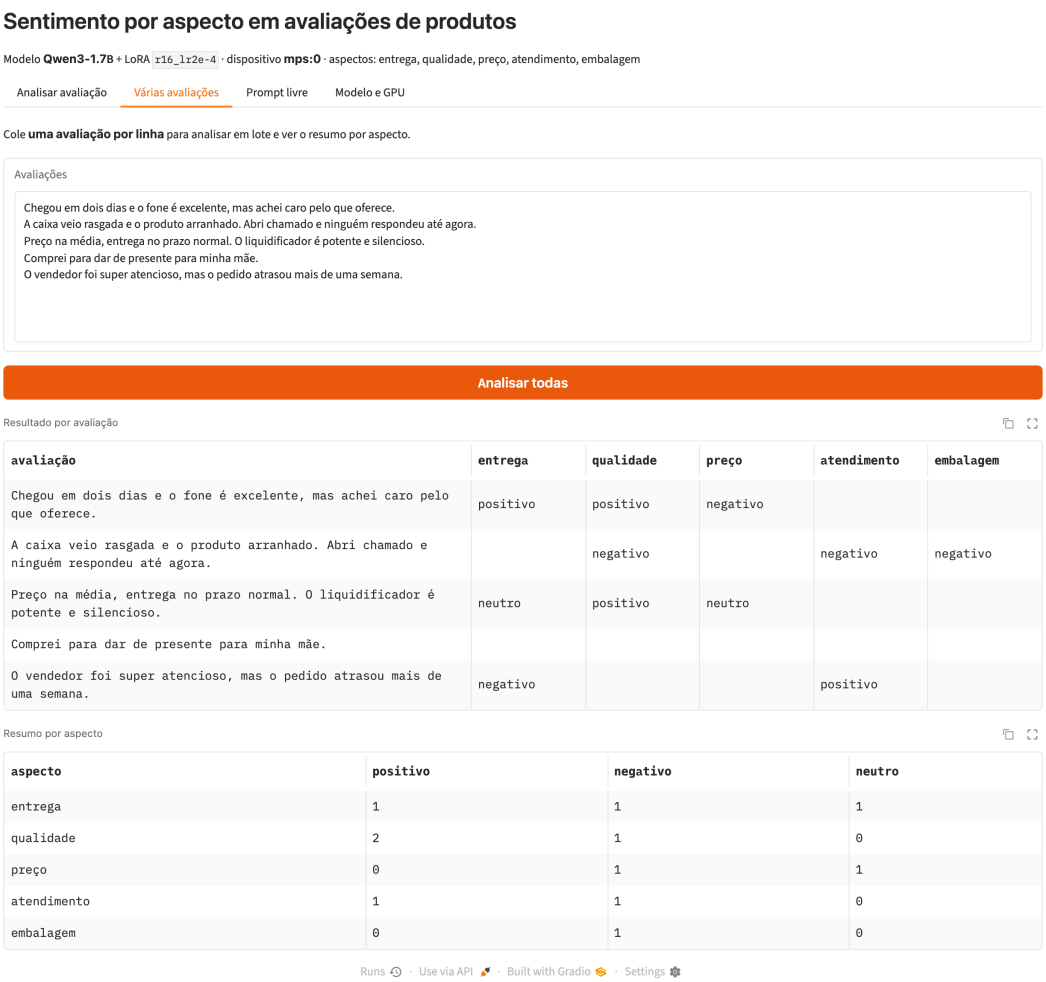
12. Frontend

Interface em **Gradio** (`app/app.py`), com quatro abas:

- **Analisar avaliação:** o usuário digita uma avaliação e vê os aspectos com sentimentos coloridos, lado a lado com a resposta do modelo **sem treino** (o LoRA é desligado na hora, sem carregar outro modelo na memória).
- **Várias avaliações:** uma por linha; mostra uma tabela por avaliação e um resumo por aspecto.
- **Prompt livre:** o usuário cria o próprio prompt (system + mensagem) e ajusta temperatura, top-p, tamanho máximo da resposta e o modo de raciocínio do Qwen3, com a resposta aparecendo em streaming.
- **Modelo e GPU:** informações da GPU e da configuração de treino do adaptador carregado.



Aba "Analisar avaliação": modelo treinado × modelo sem treino. O modelo sem treino inventa o aspecto embalagem, que não é mencionado.



Aba "Várias avaliações": análise em lote e resumo por aspecto.

13. Como executar

```
git clone https://github.com/lgustavobarre351/GENERATIVE_AI_CP2-CP3.git
cd GENERATIVE_AI_CP2-CP3
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt   # NVIDIA: instalar antes o torch com CUDA

python src/gpu_info.py            # confere a GPU
python src/gpu_monitor.py         # monitor de GPU ao vivo
python src/data.py                # gera os dados
python src/experiments.py         # baseline + 7 experimentos + comparação
python app/app.py --adapter outputs/r16_lr2e-4   # frontend em http://localhost:7860
```

Também há tarefas prontas no VSCode (*Terminal → Run Task*) e um notebook com o roteiro completo (`notebooks/projeto.ipynb`).

14. Estrutura do repositório

Arquivo	Função
src/config.py	Modelo, aspectos, prompt e detecção de GPU (CUDA > Metal/MPS > CPU)
src/data.py	Gerador do dataset sintético
src/modeling.py	Carrega modelo + LoRA, geração em lote e parser do JSON
src/train.py	Fine-tuning com LoRA (parâmetros via linha de comando)
src/evaluate.py	Métricas, matriz de confusão e respostas de cada exemplo
src/experiments.py	Grade de experimentos (treino + avaliação), retomável

Arquivo	Função
src/compare.py	Tabela e gráficos comparativos
src/gpu_info.py / gpu_monitor.py	Informações da GPU + benchmark / monitor ao vivo
app/app.py	Frontend Gradio
notebooks/projeto.ipynb	Roteiro completo para rodar no VSCode
data/teste_manual.jsonl	40 avaliações escritas à mão
outputs/r16_lr2e-4/	Adaptador LoRA do melhor modelo (33 MB)
results/	Métricas (.json), respostas (.csv), gráficos (.png) e log da execução
docs/	Guia de GPU/VSCode, prints e este relatório

15. Limitações e próximos passos

- Dados sintéticos: as frases vêm de um banco finito. O teste com frases inéditas e o teste manual medem a generalização, mas avaliações reais são mais variadas. Próximo passo: rotular uma amostra de avaliações reais.
- "Neutro" é a classe mais subjetiva ("preço razoável" é neutro ou positivo?).
- Aspectos fora da lista (cor, tamanho, garantia...) não são cobertos.
- O modelo ainda se apoia em palavras-chave em alguns casos ("atende", "entrega"); mais variedade de frases no treino deve corrigir.

16. Referências

- Qwen Team. *Qwen3 Technical Report*, 2025. https://huggingface.co/Qwen/Qwen3-1.7B
- Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*, 2021. https://arxiv.org/abs/2106.09685
- Pontiki, M. et al. *SemEval-2014 Task 4: Aspect Based Sentiment Analysis*, 2014.
- Hugging Face Transformers e PEFT. https://huggingface.co/docs
- PyTorch MPS backend. https://docs.pytorch.org/docs/stable/notes/mps.html

Apêndice A. Código-fonte completo

Cópia integral do código do repositório, para consulta caso o GitHub não esteja acessível. Versão disponível em https://github.com/lgustavobarre351/GENERATIVE_AI_CP2-CP3.

A.1 src/config.py

```python
"""Configurações centrais do projeto (modelo, aspectos, prompt e caminhos)."""
from pathlib import Path

import torch

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
OUTPUT_DIR = ROOT / "outputs"   # adaptadores LoRA treinados
RESULTS_DIR = ROOT / "results"  # métricas, gráficos e previsões

BASE_MODEL = "Qwen/Qwen3-1.7B"
SEED = 42

# Aspectos avaliados e sentimentos possíveis (a ordem define a ordem no JSON e nos gráficos)
ASPECTS = ["entrega", "qualidade", "preço", "atendimento", "embalagem"]
SENTIMENTS = ["positivo", "negativo", "neutro"]

SYSTEM_PROMPT = (
    "Você analisa avaliações de produtos escritas em português e identifica o sentimento de cada aspecto mencionado.\n"
    f"Aspectos possíveis: {', '.join(ASPECTS)}.\n"
    f"Sentimentos possíveis: {', '.join(SENTIMENTS)}.\n"
    "Responda somente com um JSON em que cada chave é um aspecto mencionado e o valor é o sentimento dele. "
    "Inclua apenas os aspectos mencionados na avaliação. Se nenhum aspecto for mencionado, responda {}.\n"
    'Exemplo: "Paguei barato, mas a tampa quebrou." -> {"qualidade": "negativo", "preço": "positivo"}'
)


def user_prompt(text: str) -> str:
    return f"Avaliação: {text}"


def get_device() -> str:
    """Escolhe a melhor GPU disponível: CUDA (NVIDIA) > MPS (Apple Metal) > CPU."""
    if torch.cuda.is_available():
        return "cuda"
    if torch.backends.mps.is_available():
        return "mps"
    return "cpu"


def get_dtype(device: str, training: bool = False) -> torch.dtype:
    if device == "cuda":
        # GPUs Ampere+ (RTX 30xx/40xx, A100) suportam bf16; a T4 usa fp16
        return torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
    if device == "mps":
        # Apple Silicon: bf16 no treino (os pesos do LoRA ficam em fp32, ver train.py)
        return torch.bfloat16 if training else torch.float16
    return torch.float32
```

A.2 src/data.py

```python
"""Gera o dataset sintético de sentimento por aspecto em avaliações de produtos.

Cada avaliação combina de 0 a 4 trechos, um por aspecto, cada trecho com um sentimento,
mais frases "de enchimento" sem aspecto. O rótulo é o JSON {aspecto: sentimento}.

Para medir generalização de verdade, as frases de cada (aspecto, sentimento) são divididas:
75% só aparecem no treino/validação e 25% só aparecem no teste. Além disso existe um teste
manual (data/teste_manual.jsonl), escrito à mão, com linguagem natural fora dos modelos.
"""
import argparse
import json
import random
import re
import unicodedata

from config import ASPECTS, DATA_DIR, SEED

PHRASES = {
    "entrega": {
        "positivo": [
            "a entrega foi super rápida", "chegou antes do prazo", "chegou em dois dias",
            "entregaram bem antes do previsto", "o pedido chegou rapidinho", "a transportadora foi muito ágil",
            "recebi no dia seguinte à compra", "o prazo de entrega foi cumprido com folga",
            "chegou muito antes do esperado", "a entrega foi rápida e sem problemas", "o frete foi rapidíssimo",
            "veio voando, chegou em 24 horas", "os correios entregaram rapidinho",
            "a entrega me surpreendeu de tão rápida", "chegou uma semana antes da data prevista",
```

```
            "a entrega foi relâmpago",
        ],
        "negativo": [
            "a entrega atrasou mais de uma semana", "demorou demais para chegar",
            "o pedido chegou com 15 dias de atraso", "a transportadora perdeu meu pacote",
            "o prazo de entrega não foi cumprido", "entregaram no endereço errado", "a entrega foi uma novela",
            "esperei quase um mês pelo pedido", "o rastreio ficou parado por dias",
            "chegou muito depois do prometido", "a entrega foi lenta demais",
            "tive que buscar na agência porque não entregaram", "remarcaram a entrega três vezes",
            "atrasaram a entrega sem nenhum aviso", "o frete demorou uma eternidade",
            "a entrega foi muito demorada",
        ],
        "neutro": [
            "a entrega foi no prazo normal", "chegou no prazo previsto, nem antes nem depois",
            "o tempo de entrega foi o padrão", "a entrega levou uns cinco dias, o normal",
            "o prazo de entrega foi o informado no site", "a entrega foi dentro do esperado",
            "chegou na data prevista", "a entrega foi comum, sem nada de especial",
            "o frete levou o tempo de sempre", "recebi na data estimada", "a entrega foi normal",
            "o prazo de entrega foi o de costume",
        ],
    },
    "qualidade": {
        "positivo": [
            "o produto é de ótima qualidade", "o material é excelente e bem resistente", "funciona perfeitamente",
            "o acabamento é impecável", "a qualidade superou minhas expectativas", "é muito bem feito e durável",
            "o som é incrível", "a tela tem cores lindas", "é muito resistente, já caiu e nada aconteceu",
            "funciona muito bem desde o primeiro dia", "é um produto de primeira linha",
            "o tecido é macio e de qualidade", "a bateria dura o dia inteiro", "está tudo funcionando perfeitamente",
            "a qualidade é excelente", "o produto é robusto e muito bem construído",
        ],
        "negativo": [
            "o produto veio com defeito", "parou de funcionar em uma semana", "o material é frágil e mal acabado",
            "veio todo arranhado", "quebrou no primeiro uso", "a qualidade é péssima",
            "o tecido desbotou na primeira lavagem", "a bateria não dura nada", "esquenta muito e desliga sozinho",
            "veio com peças faltando", "o som é muito ruim e chiado", "a costura soltou em poucos dias",
            "parece de brinquedo de tão frágil", "não funciona direito", "a tela veio trincada",
            "o produto é muito mal feito",
        ],
        "neutro": [
            "a qualidade é razoável", "o produto é ok, nada demais", "a qualidade é mediana",
            "é um produto simples, cumpre o básico", "o material é comum", "a qualidade não é boa nem ruim",
            "a qualidade é a esperada para a categoria", "funciona normalmente", "é um produto padrão",
            "o acabamento é simples", "atende o básico", "o desempenho é o normal para esse tipo de produto",
        ],
    },
    "preço": {
        "positivo": [
            "o preço estava ótimo", "comprei em promoção e valeu muito a pena", "o custo-benefício é excelente",
            "estava bem mais barato que nas outras lojas", "o preço é justo pelo que oferece",
            "paguei pouco por um produto desses", "o valor é muito em conta", "foi uma pechincha",
            "vale cada centavo", "o preço está abaixo do mercado", "o desconto foi excelente",
            "o preço foi muito camarada",
        ],
        "negativo": [
            "o preço é muito alto", "é caro demais pelo que entrega", "não vale o que custa",
            "achei o valor um absurdo", "paguei caro e me arrependi", "o preço é bem salgado",
            "o custo-benefício é ruim", "encontrei bem mais barato em outras lojas", "o valor cobrado é exagerado",
            "está caro para o que oferece", "o preço não compensa", "aumentaram o preço sem motivo",
        ],
        "neutro": [
            "o preço é o mesmo das outras lojas", "o valor é o normal de mercado", "o preço está na média",
            "não é caro nem barato", "o preço é compatível com o mercado", "paguei o preço normal",
            "o valor está dentro do esperado", "custa o mesmo que em qualquer loja",
            "o valor é o padrão para esse tipo de produto", "o preço é o de tabela",
        ],
    },
    "atendimento": {
        "positivo": [
            "o atendimento foi excelente", "o vendedor respondeu rapidinho todas as dúvidas",
            "o suporte resolveu meu problema na hora", "fui muito bem atendido", "o SAC foi atencioso e educado",
            "a loja trocou sem burocracia", "o vendedor foi super prestativo", "o pós-venda é muito bom",
            "me ajudaram em tudo pelo chat", "o atendimento foi nota dez", "responderam meu e-mail no mesmo dia",
            "a equipe de suporte foi muito simpática",
        ],
        "negativo": [
            "o atendimento é péssimo", "o SAC não resolve nada", "ninguém responde as mensagens",
            "o vendedor foi grosseiro", "fiquei horas esperando no chat", "abri três chamados e nenhum foi respondido",
            "o suporte me deixou sem resposta", "a loja se recusou a trocar", "o atendente desligou na minha cara",
            "o pós-venda é horrível", "me trataram com descaso", "prometeram retorno e nunca ligaram",
        ],
        "neutro": [
            "o atendimento foi normal", "o suporte deu uma resposta padrão", "o vendedor respondeu no tempo normal",
            "o atendimento foi comum", "falei com o SAC e foi um atendimento normal",
            "o chat respondeu dentro do horário comercial", "o atendimento foi protocolar",
            "o atendimento não foi bom nem ruim", "o atendente seguiu o roteiro de sempre",
            "a resposta do vendedor veio no prazo de costume",
        ],
    },
    "embalagem": {
```

```
        "positivo": [
            "a embalagem é muito caprichada", "veio muito bem embalado", "a embalagem é reforçada, com plástico bolha",
            "a caixa chegou intacta", "a embalagem é linda, ótima para presente", "veio tudo muito bem protegido na caixa"
,
            "embalaram com muito cuidado", "a embalagem veio perfeita", "veio em caixa resistente e lacrada",
            "capricharam na embalagem",
        ],
        "negativo": [
            "a embalagem veio rasgada", "a caixa chegou toda amassada", "veio mal embalado, sem proteção nenhuma",
            "a embalagem estava violada", "a caixa chegou molhada", "o lacre estava rompido",
            "a embalagem é frágil, sem plástico bolha", "veio solto dentro de uma caixa enorme",
            "a embalagem chegou aberta", "a caixa estava destruída",
        ],
        "neutro": [
            "a embalagem é simples", "veio na caixa padrão", "a embalagem é comum", "a embalagem é a de sempre",
            "veio numa embalagem básica", "a caixa é normal, sem nada de especial",
            "a embalagem é de papelão simples", "a embalagem é discreta",
        ],
    },
}

# Frases sem aspecto (contexto ou opinião geral): o modelo deve ignorá-las
FILLERS = [
    "Comprei para dar de presente.", "Comprei para usar no trabalho.", "É a segunda vez que compro.",
    "Minha esposa escolheu.", "Uso há duas semanas.", "Comprei na Black Friday.", "Vou usar na casa de praia.",
    "Estava precisando de um desses.", "Pesquisei bastante antes de comprar.", "Comprei pelo aplicativo.",
    "Recomendo!", "Não recomendo.", "Estou satisfeito.", "Que decepção.", "Nota 10.", "Gostei.",
    "Meu filho adorou.", "Comprei para substituir o antigo.", "Primeira compra nessa loja.", "Era o que eu queria.",
]

CONTRAST = [", mas ", ", porém ", ". Por outro lado, ", ", só que ", "; no entanto, ", ". Pena que "]
SAME = [", e ", ". ", ". Além disso, ", ", ", " e "]
N_ASPECTS_WEIGHTS = [0.05, 0.30, 0.40, 0.20, 0.05]  # probabilidade de 0, 1, 2, 3, 4 aspectos


def split_phrases(rng: random.Random):
    """Separa as frases de cada (aspecto, sentimento): 75% treino/validação, 25% só no teste."""
    train, test = {}, {}
    for aspect, by_sent in PHRASES.items():
        for sent, phrases in by_sent.items():
            shuffled = phrases[:]
            rng.shuffle(shuffled)
            cut = round(len(shuffled) * 0.75)
            train[(aspect, sent)] = shuffled[:cut]
            test[(aspect, sent)] = shuffled[cut:]
    return train, test


def strip_accents(text: str) -> str:
    return "".join(c for c in unicodedata.normalize("NFD", text) if unicodedata.category(c) != "Mn")


def make_review(rng: random.Random, bank: dict) -> dict:
    n = rng.choices(range(len(N_ASPECTS_WEIGHTS)), N_ASPECTS_WEIGHTS)[0]
    aspects = rng.sample(ASPECTS, n)
    label = {a: rng.choice(["positivo", "negativo", "neutro"]) for a in aspects}

    text = ""
    prev = None
    for aspect in aspects:
        phrase = rng.choice(bank[(aspect, label[aspect])])
        if not text:
            text = phrase
        else:
            text += rng.choice(CONTRAST if label[aspect] != prev else SAME) + phrase
        prev = label[aspect]
    if text:
        text = text[0].upper() + text[1:] + rng.choice([".", ".", ".", "!", ""])
        text = re.sub(r"([.!?] )(\w)", lambda m: m.group(1) + m.group(2).upper(), text)

    # Frases sem aspecto antes ou depois (sempre pelo menos uma se não houver aspecto)
    for _ in range(rng.choices([0, 1, 2], [0.5, 0.4, 0.1])[0] or (0 if aspects else 1)):
        filler = rng.choice(FILLERS)
        if rng.random() < 0.5 or not text:
            text = f"{filler} {text}".strip()
        else:
            text = f"{text if text[-1] in '.!' else text + '.'} {filler}"

    # Ruído de texto real: tudo minúsculo, sem acento
    if rng.random() < 0.25:
        text = text.lower()
    if rng.random() < 0.15:
        text = strip_accents(text)

    ordered = {a: label[a] for a in ASPECTS if a in label}
    return {"text": text, "label": ordered}


def generate(n: int, bank: dict, rng: random.Random, seen: set) -> list[dict]:
```

```
    rows = []
    while len(rows) < n:
        row = make_review(rng, bank)
        if row["text"] not in seen:
            seen.add(row["text"])
            rows.append(row)
    return rows


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--train", type=int, default=2000)
    ap.add_argument("--val", type=int, default=200)
    ap.add_argument("--test", type=int, default=400)
    args = ap.parse_args()

    rng = random.Random(SEED)
    train_bank, test_bank = split_phrases(rng)
    seen = set()
    splits = {
        "train": generate(args.train, train_bank, rng, seen),
        "val": generate(args.val, train_bank, rng, seen),
        "test": generate(args.test, test_bank, rng, seen),
    }
    DATA_DIR.mkdir(exist_ok=True)
    for name, rows in splits.items():
        path = DATA_DIR / f"{name}.jsonl"
        with open(path, "w") as f:
            for row in rows:
                f.write(json.dumps(row, ensure_ascii=False) + "\n")
        print(f"{name}: {len(rows)} exemplos -> {path}")
    print("\nExemplos:")
    for row in splits["train"][:5]:
        print(f"  {row['text']}\n    -> {json.dumps(row['label'], ensure_ascii=False)}")


if __name__ == "__main__":
    main()
```

A.3 src/modeling.py

```
"""Carregamento do modelo, montagem do prompt, análise por aspecto e geração livre.

Compartilhado por treino, avaliação e frontend, para que todos usem exatamente o mesmo prompt.
"""
import json
import re
import unicodedata
from threading import Thread

import torch
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer, TextIteratorStreamer

from config import ASPECTS, BASE_MODEL, SENTIMENTS, SYSTEM_PROMPT, get_device, get_dtype, user_prompt


def load_tokenizer(base_model: str = BASE_MODEL):
    tok = AutoTokenizer.from_pretrained(base_model)
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token
    return tok


def load_model(adapter: str | None = None, base_model: str = BASE_MODEL, device: str | None = None):
    """Carrega o modelo base e, opcionalmente, o adaptador LoRA treinado por cima."""
    device = device or get_device()
    tok = load_tokenizer(base_model)
    model = AutoModelForCausalLM.from_pretrained(base_model, dtype=get_dtype(device)).to(device)
    if adapter:
        model = PeftModel.from_pretrained(model, adapter)
    model.eval()
    return model, tok


def chat_prompt(tok, text: str, system: str = SYSTEM_PROMPT, thinking: bool = False) -> str:
    messages = [{"role": "system", "content": system}, {"role": "user", "content": user_prompt(text)}]
    # enable_thinking=False: o Qwen3 responde direto, sem o bloco <think> de raciocínio
    return tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True, enable_thinking=thinking)


def _norm(s: str) -> str:
    s = unicodedata.normalize("NFD", str(s).strip().lower())
    return "".join(c for c in s if unicodedata.category(c) != "Mn")


_ASPECT_KEYS = {_norm(a): a for a in ASPECTS}
_SENT_KEYS = {_norm(s): s for s in SENTIMENTS}
```

```

def parse_output(raw: str) -> dict | None:
    """Converte a resposta do modelo em {aspecto: sentimento}. Retorna None se não houver JSON válido.

    É tolerante a variações comuns do modelo base (texto antes/depois, acentos, maiúsculas),
    mas descarta aspectos e sentimentos fora da lista.
    """
    raw = re.sub(r"<think>.*?</think>", "", raw, flags=re.S)
    match = re.search(r"\{.*?\}", raw, flags=re.S)
    if not match:
        return None
    try:
        data = json.loads(match.group(0))
    except json.JSONDecodeError:
        return None
    if not isinstance(data, dict):
        return None
    out = {}
    for k, v in data.items():
        aspect, sent = _ASPECT_KEYS.get(_norm(k)), _SENT_KEYS.get(_norm(v))
        if aspect and sent:
            out[aspect] = sent
    return {a: out[a] for a in ASPECTS if a in out}


@torch.no_grad()
def analyze(model, tok, texts: list[str], batch_size: int = 16, max_new_tokens: int = 64) -> list[str]:
    """Gera a resposta (JSON em texto) para cada avaliação, em lotes e de forma determinística."""
    tok.padding_side = "left"
    outputs = []
    for i in range(0, len(texts), batch_size):
        prompts = [chat_prompt(tok, t) for t in texts[i:i + batch_size]]
        enc = tok(prompts, return_tensors="pt", padding=True, add_special_tokens=False).to(model.device)
        gen = model.generate(**enc, max_new_tokens=max_new_tokens, do_sample=False,
                             pad_token_id=tok.pad_token_id, temperature=None, top_p=None, top_k=None)
        outputs += tok.batch_decode(gen[:, enc["input_ids"].shape[1]:], skip_special_tokens=True)
    return [o.strip() for o in outputs]


def generate_stream(model, tok, system: str, prompt: str, max_new_tokens: int = 256,
                    temperature: float = 0.7, top_p: float = 0.9, thinking: bool = False):
    """Geração livre com streaming, usada na aba 'Prompt livre' do frontend."""
    messages = [{"role": "system", "content": system}, {"role": "user", "content": prompt}]
    inputs = tok.apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt",
                                     return_dict=True, enable_thinking=thinking).to(model.device)
    streamer = TextIteratorStreamer(tok, skip_prompt=True, skip_special_tokens=True)
    kwargs = dict(**inputs, streamer=streamer, max_new_tokens=max_new_tokens, pad_token_id=tok.pad_token_id)
    if temperature > 0:
        kwargs.update(do_sample=True, temperature=temperature, top_p=top_p)
    else:
        kwargs.update(do_sample=False, temperature=None, top_p=None, top_k=None)
    Thread(target=model.generate, kwargs=kwargs).start()
    text = ""
    for piece in streamer:
        text += piece
        yield text
```

A.4 src/train.py

```

"""Fine-tuning do Qwen3 com LoRA para sentimento por aspecto (resposta em JSON).

Exemplo:
    python src/train.py --run_name r16_lr2e-4 --lora_r 16 --lr 2e-4 --epochs 1
"""
import argparse
import json
import time

import pandas as pd
import torch
from datasets import Dataset
from peft import LoraConfig, get_peft_model
from transformers import (AutoModelForCausalLM, DataCollatorForSeq2Seq, Trainer, TrainerCallback,
                          TrainingArguments, set_seed)

from config import BASE_MODEL, DATA_DIR, OUTPUT_DIR, SEED, get_device, get_dtype
from modeling import chat_prompt, load_tokenizer

TARGET_MODULES = {
    "attn": ["q_proj", "k_proj", "v_proj", "o_proj"],
    "all": ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
}


class EmptyMPSCache(TrainerCallback):
    """No Apple Silicon, o cache de memória do MPS cresce ao longo do treino e o Mac começa a usar swap.
    Liberar o cache periodicamente mantém a velocidade estável."""
```

```
29
30      def on_step_end(self, args, state, control, **kwargs):
31          if state.global_step % 10 == 0:
32              torch.mps.empty_cache()
33
34
35  def parse_args(argv=None):
36      ap = argparse.ArgumentParser()
37      ap.add_argument("--run_name", required=True)
38      ap.add_argument("--base_model", default=BASE_MODEL)
39      ap.add_argument("--lora_r", type=int, default=16)
40      ap.add_argument("--lora_alpha", type=int, default=None, help="padrão: 2 * r")
41      ap.add_argument("--lora_dropout", type=float, default=0.05)
42      ap.add_argument("--target", choices=TARGET_MODULES, default="all")
43      ap.add_argument("--lr", type=float, default=2e-4)
44      ap.add_argument("--epochs", type=float, default=1)
45      ap.add_argument("--batch_size", type=int, default=4)
46      ap.add_argument("--grad_accum", type=int, default=4)
47      ap.add_argument("--max_len", type=int, default=320)
48      ap.add_argument("--no_grad_ckpt", action="store_true",
49                      help="desliga gradient checkpointing (mais rápido, usa mais memória)")
50      ap.add_argument("--train_limit", type=int, default=None, help="usar só N exemplos (teste rápido)")
51      ap.add_argument("--val_limit", type=int, default=None)
52      return ap.parse_args(argv)
53
54
55  def load_split(name: str, limit: int | None) -> pd.DataFrame:
56      df = pd.read_json(DATA_DIR / f"{name}.jsonl", lines=True)
57      return df.head(limit) if limit else df
58
59
60  def tokenize(df: pd.DataFrame, tok, max_len: int) -> Dataset:
61      """Prompt + resposta; a loss só é calculada nos tokens da resposta (o JSON)."""
62      rows = []
63      for text, label in zip(df["text"], df["label"]):
64          prompt_ids = tok(chat_prompt(tok, text), add_special_tokens=False).input_ids
65          answer = json.dumps(dict(label), ensure_ascii=False)
66          answer_ids = tok(answer, add_special_tokens=False).input_ids + [tok.eos_token_id]
67          prompt_ids = prompt_ids[-(max_len - len(answer_ids)):]
68          rows.append({
69              "input_ids": prompt_ids + answer_ids,
70              "attention_mask": [1] * (len(prompt_ids) + len(answer_ids)),
71              "labels": [-100] * len(prompt_ids) + answer_ids,
72          })
73      return Dataset.from_list(rows)
74
75
76  def main(argv=None):
77      args = parse_args(argv)
78      args.lora_alpha = args.lora_alpha or 2 * args.lora_r
79      set_seed(SEED)
80      device = get_device()
81      dtype = get_dtype(device, training=True)
82      out_dir = OUTPUT_DIR / args.run_name
83      print(f"Dispositivo: {device} | dtype: {dtype} | saída: {out_dir}")
84
85      tok = load_tokenizer(args.base_model)
86      model = AutoModelForCausalLM.from_pretrained(args.base_model, dtype=dtype)
87      model.config.use_cache = False
88      if not args.no_grad_ckpt:
89          model.gradient_checkpointing_enable(gradient_checkpointing_kwargs={"use_reentrant": False})
90          model.enable_input_require_grads()
91
92      lora = LoraConfig(r=args.lora_r, lora_alpha=args.lora_alpha, lora_dropout=args.lora_dropout,
93                        target_modules=TARGET_MODULES[args.target], task_type="CAUSAL_LM")
94      model = get_peft_model(model, lora)
95      # Pesos do LoRA em fp32 (o modelo base continua em fp16) para o treino misto ser estável
96      for p in model.parameters():
97          if p.requires_grad:
98              p.data = p.data.float()
99      model.print_trainable_parameters()
100     trainable, total = model.get_nb_trainable_parameters()
101
102     train_ds = tokenize(load_split("train", args.train_limit), tok, args.max_len)
103     val_ds = tokenize(load_split("val", args.val_limit), tok, args.max_len)
104
105     training_args = TrainingArguments(
106         output_dir=str(out_dir / "checkpoints"),
107         per_device_train_batch_size=args.batch_size,
108         per_device_eval_batch_size=args.batch_size,
109         gradient_accumulation_steps=args.grad_accum,
110         learning_rate=args.lr,
111         num_train_epochs=args.epochs,
112         lr_scheduler_type="cosine",
113         warmup_steps=0.05,  # 5% dos passos (float = proporção no transformers v5)
114         weight_decay=0.0,
115         logging_steps=10,
116         eval_strategy="steps",
117         eval_steps=0.5,  # avalia na metade e no fim do treino
```

```
        save_strategy="no",
        fp16=dtype == torch.float16,
        bf16=dtype == torch.bfloat16 and device == "cuda",  # no MPS o modelo já está em bf16, sem autocast
        report_to="none",
        seed=SEED,
        dataloader_pin_memory=device == "cuda",
    )
    trainer = Trainer(model=model, args=training_args, train_dataset=train_ds, eval_dataset=val_ds,
                      # Comprimentos múltiplos de 32: menos formatos distintos de tensor = menos recompilação na GPU
                      data_collator=DataCollatorForSeq2Seq(tok, padding=True, pad_to_multiple_of=32,
                                                           label_pad_token_id=-100),
                      callbacks=[EmptyMPSCache()] if device == "mps" else None)

    if device == "cuda":
        torch.cuda.reset_peak_memory_stats()
    t0 = time.time()
    result = trainer.train()
    train_time = time.time() - t0

    model.save_pretrained(out_dir)
    tok.save_pretrained(out_dir)
    info = {
        "run_name": args.run_name,
        "params": vars(args),
        "device": device,
        "gpu": torch.cuda.get_device_name(0) if device == "cuda" else device,
        "dtype": str(dtype),
        "trainable_params": trainable,
        "total_params": total,
        "train_examples": len(train_ds),
        "train_time_s": round(train_time, 1),
        "train_loss": round(result.training_loss, 4),
        "final_eval_loss": round(trainer.evaluate()["eval_loss"], 4),
        "peak_gpu_mem_gb": round(torch.cuda.max_memory_allocated() / 1024**3, 2) if device == "cuda" else None,
        "log_history": trainer.state.log_history,
    }
    (out_dir / "run_info.json").write_text(json.dumps(info, indent=2, ensure_ascii=False))
    print(f"Treino concluído em {train_time:.0f}s | loss {info['train_loss']} | eval_loss {info['final_eval_loss']}")
    return out_dir


if __name__ == "__main__":
    main()
```

A.5 src/evaluate.py

```
"""Avalia o modelo base (zero-shot) ou um adaptador LoRA nos conjuntos de teste.

Conjuntos:
    teste  -> 400 avaliações sintéticas com frases que NUNCA aparecem no treino
    manual -> 40 avaliações escritas à mão, com linguagem natural, gírias e erros de digitação

Exemplos:
    python src/evaluate.py --name baseline_zero_shot
    python src/evaluate.py --name r16_lr2e-4 --adapter outputs/r16_lr2e-4
"""
import argparse
import json
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
import torch

from config import ASPECTS, BASE_MODEL, DATA_DIR, RESULTS_DIR, SENTIMENTS
from modeling import analyze, load_model, parse_output

EVAL_SETS = {"teste": "test.jsonl", "manual": "teste_manual.jsonl"}


def prf(tp: int, n_pred: int, n_gold: int) -> dict:
    p = tp / n_pred if n_pred else 0.0
    r = tp / n_gold if n_gold else 0.0
    f = 2 * p * r / (p + r) if p + r else 0.0
    return {"precisao": round(p, 4), "recall": round(r, 4), "f1": round(f, 4)}


def compute_metrics(golds: list[dict], preds: list[dict | None]) -> dict:
    preds_ok = [p if p is not None else {} for p in preds]
    gold_pairs = [set(g.items()) for g in golds]
    pred_pairs = [set(p.items()) for p in preds_ok]
    tp_pairs = sum(len(g & p) for g, p in zip(gold_pairs, pred_pairs))
    tp_asp = sum(len(set(g) & set(p)) for g, p in zip(golds, preds_ok))

    # Polaridade: entre os aspectos que o modelo detectou corretamente, quantos sentimentos acertou
```

```
    cm = [[0] * len(SENTIMENTS) for _ in SENTIMENTS]
    for g, p in zip(golds, preds_ok):
        for a in set(g) & set(p):
            cm[SENTIMENTS.index(g[a])][SENTIMENTS.index(p[a])] += 1
    both = sum(sum(row) for row in cm)

    per_aspect = {}
    for a in ASPECTS:
        tp = sum(1 for g, p in zip(golds, preds_ok) if a in g and g.get(a) == p.get(a))
        per_aspect[a] = prf(tp, sum(a in p for p in preds_ok), sum(a in g for g in golds))["f1"]

    return {
        "n": len(golds),
        "json_valido": round(sum(p is not None for p in preds) / len(preds), 4),
        "acerto_exato": round(sum(g == p for g, p in zip(golds, preds_ok)) / len(golds), 4),
        "pares": prf(tp_pairs, sum(map(len, pred_pairs)), sum(map(len, gold_pairs))),
        "deteccao_aspectos": prf(tp_asp, sum(map(len, preds_ok)), sum(map(len, golds))),
        "acuracia_polaridade": round(sum(cm[i][i] for i in range(len(SENTIMENTS))) / both, 4) if both else 0.0,
        "f1_por_aspecto": per_aspect,
        "matriz_polaridade": cm,
    }


def plot_confusion(cm, title: str, path: Path):
    fig, ax = plt.subplots(figsize=(4.8, 4.2))
    ax.imshow(cm, cmap="Blues")
    ax.set_xticks(range(len(SENTIMENTS)), SENTIMENTS)
    ax.set_yticks(range(len(SENTIMENTS)), SENTIMENTS)
    ax.set_xlabel("Previsto")
    ax.set_ylabel("Real")
    ax.set_title(title, fontsize=9)
    vmax = max(max(r) for r in cm) or 1
    for i in range(len(SENTIMENTS)):
        for j in range(len(SENTIMENTS)):
            ax.text(j, i, cm[i][j], ha="center", va="center", color="white" if cm[i][j] > vmax / 2 else "black")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def main(argv=None):
    ap = argparse.ArgumentParser()
    ap.add_argument("--name", required=True)
    ap.add_argument("--adapter", default=None)
    ap.add_argument("--base_model", default=BASE_MODEL)
    ap.add_argument("--limit", type=int, default=None, help="limita o conjunto de teste sintético")
    ap.add_argument("--batch_size", type=int, default=16)
    args = ap.parse_args(argv)

    model, tok = load_model(args.adapter, args.base_model)
    run_info = json.loads((Path(args.adapter) / "run_info.json").read_text()) if args.adapter else None
    result = {
        "name": args.name,
        "adapter": args.adapter,
        "device": str(model.device),
        "train": {k: v for k, v in run_info.items() if k != "log_history"} if run_info else None,
        "sets": {},
    }
    RESULTS_DIR.mkdir(exist_ok=True)
    n_total, t_total = 0, 0.0

    for set_name, file in EVAL_SETS.items():
        df = pd.read_json(DATA_DIR / file, lines=True)
        if args.limit and set_name == "teste":
            df = df.head(args.limit)
        golds = [dict(label) for label in df["label"]]

        t0 = time.time()
        raw = analyze(model, tok, df["text"].tolist(), batch_size=args.batch_size)
        t_total += time.time() - t0
        n_total += len(df)
        preds = [parse_output(r) for r in raw]

        m = compute_metrics(golds, preds)
        result["sets"][set_name] = m
        pd.DataFrame({
            "text": df["text"],
            "esperado": [json.dumps(g, ensure_ascii=False) for g in golds],
            "resposta_modelo": raw,
            "previsto": [json.dumps(p, ensure_ascii=False) if p is not None else "JSON INVÁLIDO" for p in preds],
            "correto": [g == p for g, p in zip(golds, preds)],
        }).to_csv(RESULTS_DIR / f"{args.name}_{set_name}_predicoes.csv", index=False)

        print(f"\n== {args.name} | conjunto: {set_name} ({len(df)} exemplos, {result['device']}) ==")
        print(f"JSON válido: {m['json_valido']:.1%} | Acerto exato: {m['acerto_exato']:.1%} | "
              f"F1 pares: {m['pares']['f1']:.3f} | F1 detecção: {m['deteccao_aspectos']['f1']:.3f} | "
              f"Acurácia polaridade: {m['acuracia_polaridade']:.1%}")
        print("F1 por aspecto:", m["f1_por_aspecto"])
        if set_name == "manual":
```

```
            for text, g, p in list(zip(df["text"], golds, preds))[:40]:
                if g != p:
                    print(f"  ERRO: {text[:90]}\n        esperado={g}\n        previsto={p}")

    teste = result["sets"]["teste"]
    plot_confusion(teste["matriz_polaridade"],
                   f"{args.name} — polaridade (teste)\nF1 pares={teste['pares']['f1']:.3f}",
                   RESULTS_DIR / f"{args.name}_confusao.png")
    result["ms_por_exemplo"] = round(1000 * t_total / n_total, 1)
    (RESULTS_DIR / f"{args.name}.json").write_text(json.dumps(result, indent=2, ensure_ascii=False))
    print(f"\n{result['ms_por_exemplo']} ms/exemplo")

    del model
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    return result


if __name__ == "__main__":
    main()
```

A.6 src/experiments.py

```
"""Roda a grade de experimentos (treino + avaliação) variando os hiperparâmetros.

Cada experimento roda em um subprocesso separado para liberar a memória da GPU entre eles.

    python src/experiments.py                  # todos
    python src/experiments.py --only r16_lr2e-4 r8_lr2e-4
    python src/experiments.py --quick          # teste rápido com poucos exemplos
"""
import argparse
import subprocess
import sys
from pathlib import Path

from config import OUTPUT_DIR, RESULTS_DIR

SRC = Path(__file__).parent

# Referência: r=16, lr=2e-4, 1 época, LoRA em todas as camadas lineares.
# Cada experimento muda UM parâmetro em relação à referência.
EXPERIMENTS = {
    "r16_lr2e-4":       ["--lora_r", "16", "--lr", "2e-4", "--epochs", "1"],
    "r4_lr2e-4":        ["--lora_r", "4",  "--lr", "2e-4", "--epochs", "1"],
    "r64_lr2e-4":       ["--lora_r", "64", "--lr", "2e-4", "--epochs", "1"],
    "r16_lr2e-5":       ["--lora_r", "16", "--lr", "2e-5", "--epochs", "1"],
    "r16_lr1e-3":       ["--lora_r", "16", "--lr", "1e-3", "--epochs", "1"],
    "r16_lr2e-4_ep2":   ["--lora_r", "16", "--lr", "2e-4", "--epochs", "2"],
    "r16_lr2e-4_attn":  ["--lora_r", "16", "--lr", "2e-4", "--epochs", "1", "--target", "attn"],
}


def run(script: str, *args: str):
    cmd = [sys.executable, str(SRC / script), *args]
    print("\n$ " + " ".join(cmd), flush=True)
    if subprocess.run(cmd).returncode != 0:
        print(f"!! FALHOU: {' '.join(args)}", flush=True)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--only", nargs="*", help="nomes dos experimentos a rodar")
    ap.add_argument("--quick", action="store_true", help="200 exemplos de treino / 60 de teste")
    ap.add_argument("--skip_baseline", action="store_true")
    args = ap.parse_args()

    quick_train = ["--train_limit", "200", "--val_limit", "30"] if args.quick else []
    quick_eval = ["--limit", "60"] if args.quick else []

    if not args.skip_baseline and not (RESULTS_DIR / "baseline_zero_shot.json").exists():
        run("evaluate.py", "--name", "baseline_zero_shot", *quick_eval)

    for name in args.only or EXPERIMENTS:
        # Permite pausar e retomar: experimentos já avaliados são pulados
        if (RESULTS_DIR / f"{name}.json").exists():
            print(f"\n>> {name} já concluído, pulando", flush=True)
            continue
        run("train.py", "--run_name", name, *EXPERIMENTS[name], *quick_train)
        run("evaluate.py", "--name", name, "--adapter", str(OUTPUT_DIR / name), *quick_eval)

    run("compare.py")


if __name__ == "__main__":
    main()
```

A.7 src/compare.py

```python
"""Junta os resultados de todos os experimentos em uma tabela (Markdown) e gráficos."""
import json

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from config import ASPECTS, OUTPUT_DIR, RESULTS_DIR


def main():
    runs = []
    for path in sorted(RESULTS_DIR.glob("*.json")):
        data = json.loads(path.read_text())
        if "sets" in data:
            runs.append(data)
    if not runs:
        print("Nenhum resultado encontrado em results/")
        return
    runs.sort(key=lambda r: r["sets"]["teste"]["pares"]["f1"], reverse=True)

    lines = [
        "| Experimento | LoRA r | Camadas | LR | Épocas | Params treináveis | Tempo de treino | Loss val "
        "| JSON válido | Acerto exato | F1 pares (teste) | F1 pares (manual) | Acerto exato (manual) |",
        "|---|---|---|---|---|---|---|---|---|---|---|---|---|",
    ]
    for r in runs:
        t, te, ma = r.get("train"), r["sets"]["teste"], r["sets"]["manual"]
        if t:
            p = t["params"]
            cols = [p["lora_r"], "todas" if p["target"] == "all" else "atenção", p["lr"], p["epochs"],
                    f"{t['trainable_params'] / 1e6:.1f}M", f"{t['train_time_s'] / 60:.0f} min", t["final_eval_loss"]]
        else:
            cols = ["—"] * 7
        cols += [f"{te['json_valido']:.0%}", f"{te['acerto_exato']:.1%}", f"**{te['pares']['f1']:.3f}**",
                 f"{ma['pares']['f1']:.3f}", f"{ma['acerto_exato']:.1%}"]
        lines.append(f"| {r['name']} | " + " | ".join(str(c) for c in cols) + " |")

    table = "\n".join(lines)
    (RESULTS_DIR / "RESULTADOS.md").write_text(
        "# Resultados dos experimentos\n\n"
        "Ordenado pelo F1 dos pares (aspecto, sentimento) no teste sintético com frases inéditas.\n\n"
        f"{table}\n")
    print(table)

    names = [r["name"] for r in runs]
    x = list(range(len(runs)))

    # Gráfico 1: F1 dos pares e acerto exato, nos dois conjuntos de teste
    fig, ax = plt.subplots(figsize=(max(7, len(runs) * 1.2), 4.2))
    series = [
        ("F1 pares (teste)", [r["sets"]["teste"]["pares"]["f1"] for r in runs]),
        ("F1 pares (manual)", [r["sets"]["manual"]["pares"]["f1"] for r in runs]),
        ("Acerto exato (teste)", [r["sets"]["teste"]["acerto_exato"] for r in runs]),
    ]
    w = 0.27
    for k, (label, vals) in enumerate(series):
        ax.bar([i + (k - 1) * w for i in x], vals, width=w, label=label)
    ax.set_xticks(x, names, rotation=30, ha="right")
    ax.set_ylim(0, 1)
    ax.set_title("Comparação dos experimentos")
    ax.legend(fontsize=8)
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()
    fig.savefig(RESULTS_DIR / "comparacao.png", dpi=150)
    plt.close(fig)

    # Gráfico 2: F1 por aspecto (baseline x melhor modelo)
    best = runs[0]
    base = next((r for r in runs if r["name"] == "baseline_zero_shot"), None)
    fig, ax = plt.subplots(figsize=(7, 3.8))
    compare = [base, best] if base and base is not best else [best]
    for k, r in enumerate(compare):
        vals = [r["sets"]["teste"]["f1_por_aspecto"][a] for a in ASPECTS]
        ax.bar([i + (k - 0.5 * (len(compare) - 1)) * 0.4 for i in range(len(ASPECTS))], vals, width=0.4, label=r["name"])
    ax.set_xticks(range(len(ASPECTS)), ASPECTS)
    ax.set_ylim(0, 1)
    ax.set_title("F1 por aspecto (teste)")
    ax.legend(fontsize=8, loc="lower right")
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()
    fig.savefig(RESULTS_DIR / "f1_por_aspecto.png", dpi=150)
    plt.close(fig)

    # Gráfico 3: curvas de loss de treino
    fig, ax = plt.subplots(figsize=(7, 4))
    for r in runs:
```

```
        info_path = OUTPUT_DIR / r["name"] / "run_info.json"
        if not info_path.exists():
            continue
        hist = [h for h in json.loads(info_path.read_text())["log_history"] if "loss" in h]
        ax.plot([h["step"] for h in hist], [h["loss"] for h in hist], label=r["name"])
    ax.set_xlabel("Passo")
    ax.set_ylabel("Loss de treino")
    ax.set_yscale("log")
    ax.set_title("Curvas de loss")
    ax.legend(fontsize=8)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(RESULTS_DIR / "curvas_loss.png", dpi=150)
    plt.close(fig)
    print(f"\nTabela: {RESULTS_DIR / 'RESULTADOS.md'} | gráficos: comparacao.png, f1_por_aspecto.png, curvas_loss.png")


if __name__ == "__main__":
    main()
```

A.8 src/gpu_info.py

```
"""Mostra (e salva) as informações da GPU usada no projeto.

Rode este script no terminal/notebook do VSCode e tire o print da saída
para comprovar que a GPU está instalada e funcionando.
"""
import json
import platform
import subprocess
import time

import torch

from config import RESULTS_DIR, get_device


def gpu_info() -> dict:
    device = get_device()
    info = {
        "python": platform.python_version(),
        "sistema": f"{platform.system()} {platform.release()}",
        "torch": torch.__version__,
        "device": device,
        "cuda_disponivel": torch.cuda.is_available(),
    }
    if device == "cuda":
        props = torch.cuda.get_device_properties(0)
        info.update({
            "cuda_versao_torch": torch.version.cuda,
            "cudnn_versao": torch.backends.cudnn.version(),
            "qtd_gpus": torch.cuda.device_count(),
            "gpu_nome": props.name,
            "compute_capability": f"{props.major}.{props.minor}",
            "memoria_total_gb": round(props.total_memory / 1024**3, 2),
            "multiprocessadores": props.multi_processor_count,
            "suporta_bf16": torch.cuda.is_bf16_supported(),
        })
    elif device == "mps":
        info.update(apple_gpu_info())
    return info


def _sh(cmd: list[str]) -> str:
    return subprocess.run(cmd, capture_output=True, text=True).stdout.strip()


def apple_gpu_info() -> dict:
    """Dados da GPU Apple Silicon (Metal), via system_profiler/sysctl, sem precisar de sudo."""
    display = json.loads(_sh(["system_profiler", "SPDisplaysDataType", "-json"]) or "{}")
    gpu = (display.get("SPDisplaysDataType") or [{}])[0]
    return {
        "macos": platform.mac_ver()[0],
        "chip": _sh(["sysctl", "-n", "machdep.cpu.brand_string"]),
        "gpu_nome": gpu.get("sppci_model", "Apple GPU"),
        "gpu_nucleos": gpu.get("sppci_cores"),
        "metal": gpu.get("spdisplays_mtlgpufamilysupport", "").replace("spdisplays_metal", "Metal "),
        "backend_pytorch": "MPS (Metal Performance Shaders)",
        "mps_compilado": torch.backends.mps.is_built(),
        "memoria_unificada_gb": round(int(_sh(["sysctl", "-n", "hw.memsize"])) / 1024**3, 1),
        "memoria_max_gpu_gb": round(torch.mps.recommended_max_memory() / 1024**3, 1),
    }


def benchmark(device: str, n: int = 4096, reps: int = 20) -> float:
    """Multiplicação de matrizes na GPU, para provar que ela está realmente processando."""
    dtype = torch.float16 if device != "cpu" else torch.float32
    a = torch.randn(n, n, device=device, dtype=dtype)
```

```
    b = torch.randn(n, n, device=device, dtype=dtype)
    sync = torch.cuda.synchronize if device == "cuda" else (torch.mps.synchronize if device == "mps" else (lambda: None))
    a @ b
    sync()
    t0 = time.perf_counter()
    for _ in range(reps):
        a @ b
    sync()
    elapsed = time.perf_counter() - t0
    return round(2 * n**3 * reps / elapsed / 1e12, 2)  # TFLOPS


def main():
    info = gpu_info()
    info["benchmark_matmul_tflops"] = benchmark(info["device"])

    print("=" * 60)
    print(" INFORMAÇÕES DA GPU")
    print("=" * 60)
    for k, v in info.items():
        print(f"{k:>26}: {v}")
    print("=" * 60)

    if info["device"] == "cuda":
        print(subprocess.run(["nvidia-smi"], capture_output=True, text=True).stdout)

    RESULTS_DIR.mkdir(exist_ok=True)
    (RESULTS_DIR / "gpu_info.json").write_text(json.dumps(info, indent=2, ensure_ascii=False))


if __name__ == "__main__":
    main()
```

A.9 src/gpu_monitor.py

```
"""Monitor de GPU ao vivo no terminal (ideal para deixar aberto no VSCode durante o treino).

    python src/gpu_monitor.py              # atualiza a cada 1 s
    python src/gpu_monitor.py --log results/gpu_uso.csv

Apple Silicon: lê a utilização da GPU pelo ioreg (não precisa de sudo).
NVIDIA: usa o nvidia-smi.
"""
import argparse
import re
import shutil
import subprocess
import time
from datetime import datetime


def sample_apple() -> dict:
    out = subprocess.run(["ioreg", "-r", "-d", "1", "-c", "IOAccelerator"], capture_output=True, text=True).stdout

    def num(key):
        m = re.search(rf'"{re.escape(key)}"=(\d+)', out)
        return int(m.group(1)) if m else 0

    return {
        "gpu_%": num("Device Utilization %"),
        "renderer_%": num("Renderer Utilization %"),
        "mem_gpu_gb": num("In use system memory") / 1024**3,
        "mem_alocada_gb": num("Alloc system memory") / 1024**3,
    }


def sample_nvidia() -> dict:
    out = subprocess.run(
        ["nvidia-smi", "--query-gpu=utilization.gpu,memory.used,memory.total,temperature.gpu,power.draw",
         "--format=csv,noheader,nounits"], capture_output=True, text=True).stdout.split(",")
    util, used, total, temp, power = (float(x) for x in out[:5])
    return {"gpu_%": util, "mem_gpu_gb": used / 1024, "mem_total_gb": total / 1024, "temp_c": temp, "power_w": power}


def bar(pct: float, width: int = 30) -> str:
    filled = round(width * pct / 100)
    return "" * filled + "" * (width - filled)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--interval", type=float, default=1.0)
    ap.add_argument("--log", default=None, help="salva as amostras em CSV")
    args = ap.parse_args()

    nvidia = shutil.which("nvidia-smi") is not None
    sample = sample_nvidia if nvidia else sample_apple
    print(f"Monitorando GPU {'NVIDIA (nvidia-smi)' if nvidia else 'Apple Silicon (Metal)'} — Ctrl+C para sair\n")
```

```
    log = open(args.log, "a") if args.log else None
    try:
        while True:
            s = sample()
            now = datetime.now().strftime("%H:%M:%S")
            extra = f" | {s['temp_c']:.0f}°C {s['power_w']:.0f} W" if nvidia else ""
            print(f"\r{now}  GPU {bar(s['gpu_%'])} {s['gpu_%']:5.1f}%  | memória GPU {s['mem_gpu_gb']:5.2f} GB{extra}   ",
                  end="", flush=True)
            if log:
                log.write(f"{now}," + ",".join(f"{v:.2f}" for v in s.values()) + "\n")
                log.flush()
            time.sleep(args.interval)
    except KeyboardInterrupt:
        print()


if __name__ == "__main__":
    main()
```

A.10 app/app.py

```
"""Frontend (Gradio) para o usuário testar o modelo.

    python app/app.py --adapter outputs/r16_lr2e-4          # http://localhost:7860
    python app/app.py --adapter outputs/r16_lr2e-4 --share  # gera também um link público
"""
import argparse
import contextlib
import json
import sys
from pathlib import Path

import gradio as gr
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))
from config import ASPECTS, BASE_MODEL, SENTIMENTS, SYSTEM_PROMPT  # noqa: E402
from gpu_info import gpu_info  # noqa: E402
from modeling import analyze, generate_stream, load_model, parse_output  # noqa: E402

EXAMPLES = [
    "Chegou em dois dias e o fone é excelente, mas achei caro pelo que oferece.",
    "A caixa veio rasgada e o produto arranhado. Abri chamado e ninguém respondeu até agora.",
    "Preço na média, entrega no prazo normal. O liquidificador é potente e silencioso.",
    "Comprei para dar de presente para minha mãe.",
    "O vendedor foi super atencioso, mas o pedido atrasou mais de uma semana.",
]

COLORS = {"positivo": "#1a9e5c", "negativo": "#d6453d", "neutro": "#8a8f98"}
ICONS = {"positivo": "▲", "negativo": "▼", "neutro": "●"}


def render(raw: str, title: str) -> str:
    """Mostra o resultado como uma lista de aspectos coloridos + a resposta crua do modelo."""
    parsed = parse_output(raw)
    if parsed is None:
        body = "<p style='color:#d6453d'><b>Resposta fora do formato JSON</b></p>"
    elif not parsed:
        body = "<p style='opacity:.7'>Nenhum aspecto mencionado.</p>"
    else:
        rows = "".join(
            f"<tr><td style='padding:6px 12px 6px 0'><b>{a}</b></td>"
            f"<td><span style='background:{COLORS[s]};color:white;padding:3px 10px;border-radius:12px'>"
            f"{ICONS[s]} {s}</span></td></tr>"
            for a, s in parsed.items())
        body = f"<table>{rows}</table>"
    return (f"<div style='padding:4px'><h3 style='margin:0 0 8px'>{title}</h3>{body}"
            f"<p style='margin-top:10px;font-size:12px;opacity:.7'>Resposta do modelo: <code>{raw}</code></p></div>")


def build_ui(model, tok, adapter: str | None, base_model: str = BASE_MODEL):
    has_adapter = adapter is not None

    def adapter_ctx(use_adapter: bool):
        # Com PEFT dá para desligar o LoRA e comparar com o modelo base sem carregar outro modelo na memória
        if has_adapter and not use_adapter:
            return model.disable_adapter()
        return contextlib.nullcontext()

    def run_single(text: str, compare: bool):
        if not text.strip():
            raise gr.Error("Digite uma avaliação.")
        ft = render(analyze(model, tok, [text])[0], "Modelo treinado (LoRA)" if has_adapter else "Modelo base")
        base = ""
        if has_adapter and compare:
            with adapter_ctx(False):
                base = render(analyze(model, tok, [text])[0], "Modelo base (sem treino)")
        return ft, base
```

```
68
69      def run_batch(texts: str):
70          reviews = [t.strip() for t in texts.splitlines() if t.strip()]
71          if not reviews:
72              raise gr.Error("Cole uma avaliação por linha.")
73          preds = [parse_output(r) or {} for r in analyze(model, tok, reviews)]
74          table = pd.DataFrame([{"avaliação": t, **{a: p.get(a, "") for a in ASPECTS}} for t, p in zip(reviews, preds)])
75          summary = pd.DataFrame([{"aspecto": a, **{s: sum(p.get(a) == s for p in preds) for s in SENTIMENTS}}
76                                  for a in ASPECTS])
77          return table, summary
78
79      def run_generate(system, prompt, max_tokens, temperature, top_p, thinking, use_adapter):
80          if not prompt.strip():
81              raise gr.Error("Digite um prompt.")
82          with adapter_ctx(use_adapter):
83              yield from generate_stream(model, tok, system, prompt, int(max_tokens), temperature, top_p, thinking)
84
85      run_info = {}
86      if has_adapter and (Path(adapter) / "run_info.json").exists():
87          run_info = json.loads((Path(adapter) / "run_info.json").read_text())
88          run_info.pop("log_history", None)
89
90      with gr.Blocks(title="Sentimento por aspecto") as demo:
91          gr.Markdown(
92              "# Sentimento por aspecto em avaliações de produtos\n"
93              f"Modelo **{base_model.split('/')[-1]}** "
94              f"{'+ LoRA `' + Path(adapter).name + '`' if has_adapter else '(sem fine-tuning)'} · "
95              f"dispositivo **{model.device}** · aspectos: {', '.join(ASPECTS)}"
96          )
97          with gr.Tab("Analisar avaliação"):
98              with gr.Row():
99                  with gr.Column():
100                     text = gr.Textbox(label="Avaliação do produto", lines=5,
101                                       placeholder="Ex.: Chegou rápido, mas veio arranhado.")
102                     compare = gr.Checkbox(label="Comparar com o modelo base (sem treino)", value=has_adapter,
103                                           visible=has_adapter)
104                     btn = gr.Button("Analisar", variant="primary")
105                     gr.Examples(EXAMPLES, inputs=text)
106                 with gr.Column():
107                     out_ft = gr.HTML()
108                     out_base = gr.HTML()
109             btn.click(run_single, [text, compare], [out_ft, out_base])
110             text.submit(run_single, [text, compare], [out_ft, out_base])
111
112         with gr.Tab("Várias avaliações"):
113             gr.Markdown("Cole **uma avaliação por linha** para analisar em lote e ver o resumo por aspecto.")
114             texts = gr.Textbox(label="Avaliações", lines=8, value="\n".join(EXAMPLES))
115             batch_btn = gr.Button("Analisar todas", variant="primary")
116             table = gr.Dataframe(label="Resultado por avaliação", wrap=True,
117                                  column_widths=["45%"] + ["11%"] * len(ASPECTS))
118             summary = gr.Dataframe(label="Resumo por aspecto")
119             batch_btn.click(run_batch, texts, [table, summary])
120
121         with gr.Tab("Prompt livre"):
122             gr.Markdown("Crie seu próprio prompt e converse com o modelo.")
123             with gr.Row():
124                 with gr.Column():
125                     system = gr.Textbox(label="Prompt de sistema", value=SYSTEM_PROMPT, lines=6)
126                     prompt = gr.Textbox(label="Seu prompt", lines=5,
127                                         placeholder="Ex.: Avaliação: A bateria dura pouco, mas a câmera é ótima.")
128                     with gr.Accordion("Parâmetros de geração", open=False):
129                         max_tokens = gr.Slider(8, 2048, value=256, step=8, label="Máx. de tokens")
130                         temperature = gr.Slider(0, 1.5, value=0.7, step=0.05, label="Temperatura (0 = determinístico)")
131                         top_p = gr.Slider(0.1, 1, value=0.9, step=0.05, label="Top-p")
132                         thinking = gr.Checkbox(label="Modo raciocínio do Qwen3 (<think>)", value=False)
133                         use_adapter = gr.Checkbox(label="Usar adaptador LoRA", value=has_adapter,
134                                                   visible=has_adapter)
135                     gen_btn = gr.Button("Gerar", variant="primary")
136                 with gr.Column():
137                     answer = gr.Textbox(label="Resposta", lines=16)
138             gen_btn.click(run_generate, [system, prompt, max_tokens, temperature, top_p, thinking, use_adapter],
139                           answer)
140
141         with gr.Tab("Modelo e GPU"):
142             gr.JSON(value=gpu_info(), label="GPU")
143             if run_info:
144                 gr.JSON(value=run_info, label="Configuração do treino")
145     return demo
146
147
148 def main():
149     ap = argparse.ArgumentParser()
150     ap.add_argument("--adapter", default=None, help="pasta do adaptador LoRA (ex.: outputs/r16_lr2e-4)")
151     ap.add_argument("--base_model", default=BASE_MODEL)
152     ap.add_argument("--share", action="store_true", help="gera link público")
153     ap.add_argument("--port", type=int, default=7860)
154     args = ap.parse_args()
155
156     model, tok = load_model(args.adapter, args.base_model)
```

```
157      build_ui(model, tok, args.adapter, args.base_model).queue().launch(share=args.share, server_port=args.port)
158
159
160  if __name__ == "__main__":
161      main()
```

A.11 requirements.txt

```
1  # Mac (Apple Silicon): o torch do PyPI já vem com suporte a Metal (MPS).
2  # PC com NVIDIA: instale antes o torch da versão CUDA certa (ver docs/GPU_VSCODE.md).
3  torch>=2.4
4  transformers>=5.0,<6
5  peft>=0.17
6  accelerate>=1.0
7  datasets>=3.0
8  pandas
9  matplotlib
10 gradio>=5
11 ipykernel
```